LabelAI

AI-Assisted Medical Image Annotation Platform

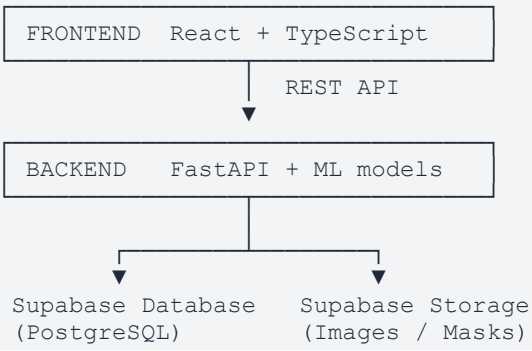
Tech stack, team structure, and 8-week MVP plan for a 3-person team

> **Core workflow:** Upload medical image → AI predicts → human corrects → save → export dataset.

1. Final Tech Stack

Layer	Technologies
Frontend	React + TypeScript, Vite, Tailwind CSS, Canvas / SVG, Zustand, Axios / Fetch
Backend	Python, FastAPI, Pydantic, SQLAlchemy / Supabase client
ML	PyTorch, SAM 3, YOLO, OpenCV
Database	Supabase Database (PostgreSQL)
Storage	Supabase Storage (images / masks)

Architecture overview



2. Tools You'll Use

Category	Tools
Development	VS Code, Git, GitHub, GitHub Issues, GitHub Projects, Python, Node.js, npm, Docker, Postman / Insomnia
Frontend	React, TypeScript, Vite, Tailwind CSS, Zustand, HTML Canvas, SVG
Backend	Python, FastAPI, Pydantic, Uvicorn, SQLAlchemy or Supabase Python client

Category	Tools
Database / Infrastructure	Supabase: PostgreSQL database, Authentication, Storage, Row Level Security, API / dashboard
ML / CV	PyTorch, OpenCV, NumPy, YOLO, SAM 3
Testing	pytest, Vitest, React Testing Library, Postman
Deployment	Docker, GitHub Actions, Vercel / Cloudflare (frontend), AWS / Render / Railway etc. (backend), Supabase (DB / storage)

3. Frontend Skills (Member 1)

Member 1 should become comfortable with:

- HTML, CSS, JavaScript, TypeScript
- React, React hooks, and component architecture
- State management
- REST APIs
- Canvas and SVG
- Mouse events
- Coordinate transformations
- Zoom / pan
- File uploads

> **Important:** Coordinate transformation is extremely important. Every mouse position on the displayed image must be mapped back to the original image's coordinates.

Example

```
Original image       1920 × 1080
      ↓ resize
Displayed image       960 × 540
      ↓
Mouse coordinate     (300, 200)
      ↓
Original coordinate  (600, 400)
```

4. Backend Skills (Member 2)

Member 2 should learn:

- Python, FastAPI, Pydantic
- REST APIs, HTTP, and JSON
- API validation and error handling
- Authentication
- Asynchronous programming basics
- Database queries
- File handling
- API testing

Example endpoints

Method	Endpoint
POST	/projects
GET	/projects
POST	/images
GET	/images/{id}
POST	/annotations
PUT	/annotations/{id}
DELETE	/annotations/{id}
POST	/predict

5. Supabase Skills

At least Member 2 should understand:

- Tables, primary keys, and foreign keys
- SQL and relationships
- Authentication
- Row Level Security and database policies
- Storage buckets
- Signed URLs

Proposed database hierarchy

```
users
  └── projects
        └── datasets
              └── images
                    └── annotations
```

6. ML / CV Skills (Member 3)

Member 3 should focus on:

- NumPy, OpenCV, PyTorch
- Image preprocessing
- Segmentation, bounding boxes, masks, and polygons
- YOLO inference and SAM 3 inference
- Model loading and GPU inference
- Converting predictions into the annotation format

The critical interface

```
ML model
   ↓
prediction
   ↓
standard annotation format
   ↓
frontend
```

Example prediction

```
{
  "type": "segmentation",
  "class": "vessel",
  "confidence": 0.91,
  "points": [...]
}
```

7. Git / GitHub Skills (Everyone)

All three members should know these commands:

```
git clone
git checkout -b
git add
git commit
git pull
git push
```

And these concepts:

- Branches and pull requests
- Code reviews
- Merge conflicts
- Issues
- .gitignore and README
- Environment variables

> **Rule:** Don't have everyone working directly on main.

Use frontend, backend, and ml as conceptual areas, with feature branches off main:

```
main
  ├── feature/image-viewer      (frontend)
  ├── feature/annotation-api    (backend)
  └── feature/sam3-inference    (ml)
```

8. Team Structure

Member	Role	Owns
Member 1	Frontend / Annotation Engine	React, TypeScript, Canvas/SVG, UI, annotation interaction, zoom/pan, drawing, undo/redo
Member 2	Backend / Supabase	FastAPI, Supabase, database, authentication, file uploads, REST API, projects/datasets, deployment

Member	Role	Owns
Member 3	ML / Computer Vision	PyTorch, YOLO, SAM 3, OpenCV, prediction API, mask processing, model inference, AI-assisted annotation

Note: Everyone should understand the full architecture, not just their own area.

9. 8-Week Plan

Week 1 — Architecture + Setup

Everyone establishes: Git/GitHub workflow, repository structure, API conventions, JSON annotation format, development environments, and Docker basics.

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Set up React, TypeScript, Vite, Tailwind • Build a basic Dashboard with Projects and Dataset views	• Set up FastAPI, Supabase, and environment variables • Create tables: projects, datasets, images, annotations	• Set up PyTorch, OpenCV, YOLO, SAM 3 • Test models independently

End of week: Everyone can run the frontend, backend, Supabase, and the ML environment.

Week 2 — Image Management

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Image viewer • Image list • Image selection • Basic toolbar • Image loading	• POST / GET /projects • POST / GET /datasets • POST / GET /images • Connect Supabase Storage	• Image preprocessing • Resizing • Coordinate conversion utilities • YOLO inference prototype • SAM 3 inference prototype

End of week: Create project → Upload images → See images in browser.

Week 3 — Bounding Box Annotation

This is the **first major milestone**.

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Rectangle tool (mouse down / move / up) • Select, move, resize, and delete annotations • Zoom • Pan	• POST /annotations • GET /annotations/{image_id} • PUT /annotations/{id} • DELETE /annotations/{id}	• Define the standardized annotation format (see example below)

Example bounding box format

```
{
  "type": "bbox",
  "class_id": 2,
  "x": 100,
  "y": 200,
  "width": 300,
  "height": 150
}
```

> **End of week:** You can manually label images.

Week 4 — Polygons + Segmentation

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Polygon tool and points • Vertex movement • Polygon selection, edit, delete • Then: mask visualization, brush • Eraser (if time allows)	• Database support for bbox, polygon, and mask annotations	• SAM 3 → prediction → mask • Polygon ↔ mask conversion

> **End of week:** Bounding boxes + polygons + segmentation masks all working.

Week 5 — AI-Assisted Labeling

This is where the project becomes much more interesting.

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Buttons: AI Detect, AI Segment, Accept, Reject, Edit • Display AI predictions with a yellow / dashed outline • Display human annotations with a solid outline	• Build POST /predict (request and response below)	• Integrate YOLO and SAM 3 into one pipeline (below)

POST /predict — request

```
{
  "image_id": 123,
  "model": "sam3"
}
```

POST /predict — response

```
{
  "predictions": [...]
}
```

Inference pipeline

```
Image → YOLO → object detection → SAM 3 → segmentation → prediction
```

> **End of week:** Upload image → AI predicts → Review → Correct → Save.

Week 6 — Dataset Management + Export

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Annotation filtering • Class selector • Image status • Next / previous image • Keyboard shortcuts • Undo / redo	• Dataset statistics: total images, labeled, unlabeled, reviewed • Export API	• Exporters: YOLO, COCO, JSON, PNG masks

> **End of week:** Take 1,000 images and produce a YOLO dataset, a COCO dataset, and segmentation masks.

Week 7 — Polish + Testing

> **Focus:** This week is not for adding random features.

Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• UI polish • Performance • Keyboard shortcuts • Responsive layout • Annotation bugs	• Authentication and permissions • Security • API validation • Database policies • Error handling	• Inference performance • GPU memory • Batching • Model caching • Prediction reliability

Everyone — testing

Area	Tool
Frontend	Vitest
Backend	pytest
API	Postman

Week 8 — Deployment + Demo

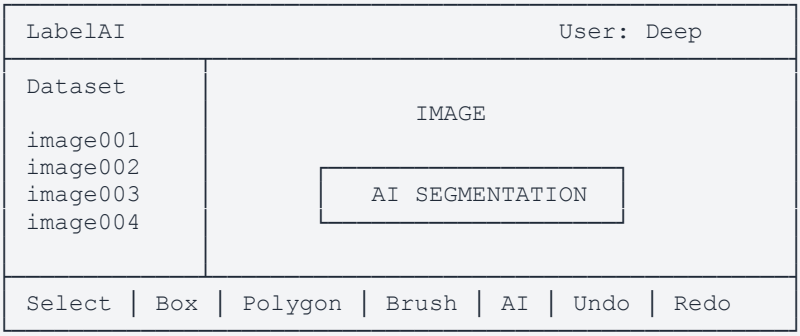
Member 1 · Frontend	Member 2 · Backend	Member 3 · ML / CV
• Deploy frontend • Polish the full flow: Login → Dashboard → Project → Dataset → Labeling interface	• Deploy FastAPI, Supabase, and Storage • Set up environment variables, CORS, HTTPS, authentication, logging	• Deploy and test YOLO and SAM 3 • Make sure model inference works reliably

Everyone creates:

- README and setup instructions
- Architecture diagram
- API documentation
- Demo dataset
- Screenshots and demo video

10. What You Should Have at the End

Your MVP should look roughly like this:



MVP feature checklist

Area	Features
Annotation	Bounding boxes, polygons, segmentation masks, multiple classes, edit / delete / move, undo / redo, zoom / pan
Dataset	Projects, datasets, image upload, image browser, label status, class management, dataset statistics
AI	YOLO detection, SAM 3 segmentation, AI predictions, human correction, accept / reject
Export	YOLO, COCO, JSON, PNG masks

11. Skills by Person

★ = *depth of skill needed (1–5)* ○ = *not needed*

Skill	M1 · Frontend	M2 · Backend	M3 · ML
Python	○	★★★	★★★★★
JavaScript	★★★★	★	★
TypeScript	★★★★★	★	★
React	★★★★★	★	○
Canvas / SVG	★★★★★	○	★
FastAPI	★★	★★★★★	★★★
Supabase	★	★★★★★	★
SQL	★★	★★★★	★★
OpenCV	★★	○	★★★★★
PyTorch	○	○	★★★★★
YOLO	○	○	★★★★★

Skill	M1 · Frontend	M2 · Backend	M3 · ML
SAM 3	○	★	★★★★★
Git	★★★	★★★	★★★
Docker	★★	★★★★	★★★
Testing	★★★	★★★	★★★
AWS / Deployment	★	★★★★	★★

12. Don't Try to Build These in 8 Weeks

Avoid scope creep. Do **not** initially build:

- Video annotation
- 3D annotation
- Collaborative real-time editing
- Complex RBAC
- Billing
- Marketplace
- Mobile app
- Advanced active learning
- Custom model training platform
- Dozens of annotation formats
- A full Label Studio replacement

Those can come later.

> **Your 8-week MVP should nail one workflow:** Upload medical image → AI predicts → human corrects → save → export dataset. That is a realistic and impressive 3-person project.
>
> **Key recommendation:** Design the internal annotation schema from day one so that bbox, polygon, mask, YOLO predictions, and SAM 3 predictions all use one common representation. That will save you a lot of pain in weeks 4–8.